SK네트웍스 **Family AI** 과정 **10**기

모델배포 개발된 LLM 연동 웹 애플리케이션

산출물 단계	모델배포
평가 산출물	LLM 연동 웹 애플리케이션
제출 일자	2025.07.04
깃허브 경로	https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN10-FINAL-3Team.git
작성 팀원	배민경, 김현수, 문승기

1. 프로젝트 개요

- 프로젝트명: **LawMate**
- 작성일: 2025.07.02
- 작성자: 배민경, 김현수

2. 시스템 개요

- 목표
  - 이 애플리케이션은 DB(RDB, VectorDB)와 LLM모델을 연동하여 등록된 사건에 대한 소송 전략을 제공하며, 판례 검색이나 유사 판례 비교, 판례 내용에 관한 질문에 대한 응답을 챗봇 형태로 제공하여 변호사와 법률 사무원의 업무 보조로 활용될 수 있도록 합니다.
- 기능:
  - 사건 등록 팀 추천
  - 판례 챗봇 기능
  - 사건 전략 추천 생성

3. 시스템 아키텍처

- 백엔드:
  - 언어: Python
  - 프레임워크: Django / FastAPI
  - LLM 연동: OpenAI GPT-4o, Runpod vLLM (Gemma3-1b-파인튜닝 모델)
  - 벡터 데이터베이스: FAISS
  - 관계형 데이터베이스: MySQL
  - 기술 스택:
    - API 관리:
      - Django REST Framework
      - FastAPI
      - CORS 설정으로 크로스 도메인 통신 지원
    - 보안:
      - JWT 인증
      - CSRF 토큰 보호
      - HttpOnly 쿠키 기반 토큰 저장
- 프론트엔드:
  - AJAX 요청을 통한 비동기 통신
  - Django 템플릿 엔진 기반 서버사이드 렌더링
  - TailwindCSS

4. 데이터베이스와 LLM 연동

4.1 사건 등록 시 팀 추천

- 연동 목적
  - 사건 등록 시 `e_description` 같은 사건 본문에서 핵심 쟁점과 맥락을 추출하고, 각 팀의 유사 사건 처리 전문성과 현재 업무 부하 를 비교하여 가장 적합한 팀(또는 동점 팀들)을 LLM의 판단과 로직을 결합해 자동 추천
  - 단순 Rule-Based가 아니라 벡터 검색, SQL 통계, AI 평가, LLM 설명을 묶어 추천팀의 정량성(수치 기반), 정성성(LLM 요약)을 함께 제공하는 목적.
- 프롬프트 (사건 본문 요약용)
  - `e_description`에서 사건 흐름과 쟁점, 당사자 주장 등 사건 이해에 핵심적인 2~3문장만 발췌
- 프롬프트 (추천 사유 생성용)
  - 팀별 `norm_sim`과 `MinMax`를 비교하여 왜 이 팀(또는 동점 팀)이 추천되었는지 상대 비교로 설명.
  - 단일 팀이 아니라 동점 팀이 있을 때도 각 팀의 강점을 간결히 설명.

예시 코드: 사건 등록 시 팀 추천 (사건 본문 요약용)

```python
def gpt_summarize(text: str) -> str:
    """
    사건 설명문을 LLM으로 요약하여 핵심 문장 2~3개 발췌
    """
    prompt = f"""
    다음은 민사소송 사건의 설명입니다.

    - 이 설명에서 **사건의 주요 흐름과 쟁점, 당사자 주장, 사실관계**를 가장 잘 드러내는 핵심 문장 2~3개만 골라주세요.
    - 반드시 **원문에서 발췌된 구체적 설명 문장**만 사용하세요.
    - 각 문장은 사건을 이해하는 데 도움이 되어야 하며, 단순 키워드가 아닌 **사건의 맥락**을 보여주는 설명이어야 합니다.
    - 각 문장은 텍스트로만 이루어져야 합니다.

    사건 설명:
    {text}

    핵심 문장 2~3개:
    """.strip()

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.3,
    )
    return response.choices[0].message.content.strip()
```

- 프롬프트 최적화:
  - 번호, 따옴표 등을 후처리로 제거하여 RAG 검색 정확도를 높임.
  - GPT 프롬프트에서 길이 제한과 예시로 일관된 스타일 확보.

예시 코드: 사건 등록 시 팀 추천 (추천 사유 생성용)

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    temperature=0.3,
    messages=[
        {
            "role": "system",
            "content": (
                "너는 법무법인 사건 배정 추천 시스템의 AI 설명 노드야. "
                "아래 details는 모든 팀의 점수와 부하 정보야.\n"
                "- norm_sim는 전문성을 나타냄 (높을수록 좋음)\n"
                "- MinMax는 업무량을 나타냄 (높을수록 적은 업무)\n"
                "추천된 팀들이 다른 팀보다 어떤 점에서 상대적으로 적합한지 "
                "간략히 비교해서 50자 이내로 설명해라."
            )
        },
        {
            "role": "user",
            "content": (
                f"추천팀들: [{team_list_str}], 점수: {score:.2f}\n"
                f"모든 팀 세부 점수: {details}\n"
                f"이 정보를 바탕으로 추천 사유를 작성해줘.\n"
                f"각 팀의 norm_sim과 MinMax를 고려하여, 왜 이 팀들이 추천되었는지 자세하게 설명하되,\n"
                "50자 이내로 요약해줘.\n"
            )
        }
    ]
)
```

- 프롬프트 최적화:
  - 디버깅 로깅으로 입력값 상태 확인.
  - norm_sim, MinMax 점수의 의미를 제공해 판단에 도움을 주도록 수정.
  - 동점 처리에 대비해 recommended_team은 리스트로 받아 안전성 확보.

4.2 사건 전략 추천 생성

- 연동 목적
  - 사건 등록 시 사건에 대한 정보를 기반으로 해당 사건을 진행함에 있어 변호 전략에 대한 인사이트를 줄 수 있도록 응답을 생성하고 데이터베이스에 저장할 수 있도록 연동
- 프롬프트
  - 소송 중 변호 할 대상에 따라 정해진 형식으로 답변이 나오도록 유도하여 설계되었습니다.

예시 코드: 사건 전략 추천 생성

```python
# 요청 프롬프트 생성 함수
def get_request_prompt(request):
    if request.client_role == "피고":
        system_prompt = f"""다음 사건에 대한 {request.client_role} 전략을 제시해주세요.
각 양식마다 줄바꿈을 사용해주세요.
출력 양식:
### 전략 방향성:
- ...
### 방어 논리:
- ...
### 참고 사항:
- ...
"""
    elif request.client_role == "원고":
        system_prompt = f"""다음 사건에 대한 {request.client_role} 전략을 제시해주세요.
각 양식마다 줄바꿈을 사용해주세요.
출력 양식:
### 예상 피고 주장:
- ...
### 전략 방향성:
- ...
### 참고 자료:
- ...
### 참고 사항:
- ...
"""
```

```python
@router.post("/add-strategy/", response_model=EventResponse)
async def analyze_case(case: EventRequest):
    """
    사건 등록 시 sLLM 모델을 통해 전략 생성
    Args:
        case: 사건 정보
            EventRequest={
                client_role = 원고/피고,
                e_description = 사건 내용,
                claim_summary = 청구 내용,
                event_file = 증거자료
            }
    Returns:
        EventResponse = {
            result = 전략 생성 결과
        }
    """
    try:
        logger.info(f"분석 요청 받음: {case.client_role}")
        # 시스템 프롬프트와 유저 프롬프트 생성
        system_prompt, user_prompt = get_request_prompt(case)
        # LLM 모델을 통해 전략 생성
        strategy = generate_strategy(system_prompt, user_prompt)
        logger.info(f"전략 생성 결과: {strategy}")
        return EventResponse(result=strategy)
    except Exception as e:
        logger.error(f"분석 중 오류 발생: {str(e)}")
        raise HTTPException(status_code=500, detail=f"분석 중 오류가 발생했습니다: {str(e)}")
```

4.3 판례 챗봇 기능

- 연동 목적
  - 유사한 판례 또는 특정 메타정보를 기반으로 판례를 조회하고, 이에 대한 질문하기 위해 LLM 모델과 벡터 DB와 RDB에 저장된 정보와 연동하여 응답을 생성
- 프롬프트
  - 챗봇에게 질문을 하였을 때 질문의 의도를 분류하고, 그 의도에 맞는 응답을 유도하도록 설계되었습니다.

예시 코드: 판례 챗봇 기능

```python
def classify_query(query: str, context: str = "search") → dict:
    """
    입력 쿼리를 분류하는 통합 함수

    Args:
        query: 사용자 쿼리
        context: 분류 컨텍스트 ("search" 또는 "qa")

    Returns:
        dict: 분류 결과
    """

    if context == "search":
        # 일반 검색 분류 (조건기반/유사도기반/일반질문)
```

5. 예외 처리 및 보안

5.1 예외 처리

- 예상치 못한 상황에 대한 예외 처리:
  - API 요청이 실패하거나, 데이터베이스가 응답하지 않을 때 적절한 오류 메시지를 사용자에게 표시
  - 예외 처리 코드 예시:

```
try:
    client = OpenAI(
        base_url=API_URL,
        api_key=API_KEY
    )
    logger.info(f"OpenAI 클라이언트 초기화 완료: {RUNPOD_API_URL}")
except Exception as e:
    logger.error(f"OpenAI 클라이언트 초기화 실패: {str(e)}")
```

- 보안:
  - RUNPOD_API_URL과 RUNPOD_API_KEY와 OPENAI_API_KEY와 같은 데이터는 환경 변수를 사용하여 코드 상으로 노출되지 않도록 합니다.
  - 환경 변수 예시:

```
import os

MYSQL_USER = os.getenv("MYSQL_USER")
MYSQL_PWD = os.getenv("MYSQL_PWD")
MYSQL_HOST = os.getenv("MYSQL_HOST")
MYSQL_PORT = os.getenv("MYSQL_PORT", "3306")
MYSQL_DB = os.getenv("MYSQL_DB")

OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

RUNPOD_API_URL = os.getenv("RUNPOD_API_URL")
RUNPOD_API_KEY = os.getenv("RUNPOD_API_KEY")
```

5.2 보안 관리

- 보안에 민감한 정보 (예: API 키, 데이터베이스 연결 정보 등)는 Runpod의 환경변수에 별도로 저장하여 관리합니다.
- 환경 변수를 사용하여 코드에 민감한 정보를 노출시키지 않도록 관리합니다.

6. 코드 모듈화 및 주석

6.1 모듈화

- 각 기능을 별도의 모듈로 분리하여 가독성을 좋게 함으로써 유지보수를 용이하게 합니다.
  - schemas 폴더를 통해 각 기능별로 request, response에 대한 스키마 정의를 구분지어 저장
  - agent/chatbot 폴더 중 classify_llm.py, reds_query.py, vector_search.py를 통해 챗봇의 기능별로 구분지어 저장

6.2 주석

```
@router.post("/add-strategy/", response_model=EventResponse)
async def analyze_case(case: EventRequest):
    """
    사건 등록 시 sLLM 모델을 통해 전략 생성
    Args:
        case: 사건 정보
            EventRequest={
                client_role = 원고/피고,
                e_description = 사건 내용,
                claim_summary = 청구 내용,
                event_file = 증거자료
            }
    Returns:
        EventResponse = {
            result = 전략 생성 결과
        }
    """
    try:
        logger.info(f"분석 요청 받음: {case.client_role}")
        # 시스템 프롬프트와 유저 프롬프트 생성
        system_prompt, user_prompt = get_request_prompt(case)
        # LLM 모델을 통해 전략 생성
        strategy = generate_strategy(system_prompt, user_prompt)
        logger.info(f"전략 생성 결과: {strategy}")
        return EventResponse(result=strategy)
    except Exception as e:
        logger.error(f"분석 중 오류 발생: {str(e)}")
        raise HTTPException(status_code=500, detail=f"분석 중 오류가 발생했습니다: {str(e)}")
```